Autonomous Arduino Car with Line Following and Obstacle Detection

Naved Afridi Rashiq, Amina Boudali, Tanseem Farwah, Tzuen Lin

1. Introduction and Objectives

This project focuses on the design and prototyping of an Arduino-based autonomous driving system. The system is implemented as a small robotic car that follows a black line using infrared sensors and detects obstacles using ultrasonic sensors. Based on sensor input, the controller adjusts the motor movement so that the car can move forward, turn left or right, stop when the line is lost, avoid obstacles, and respond with a 180-degree turn when required.

The main objectives of the project are to model the autonomous car system using SysML diagrams, define the main requirements for line following and obstacle detection, simulate the vehicle logic in Tinkercad before hardware implementation, build the Arduino-based prototype with sensors, motors, battery, and 3D-printed holders, and test the line-following, obstacle-avoidance, and 180-degree turning behavior.

2. System Design and Engineering Approach

The project followed a model-based systems engineering approach. Instead of directly starting with wiring and programming, the system was first analyzed and designed using SysML. This made it possible to connect the project objective with the required functions, hardware components, control behavior, and later prototype implementation. The SysML model includes a requirement diagram, block definition diagram, internal block diagram, use case diagram, sequence diagram, activity diagram, and package diagram.

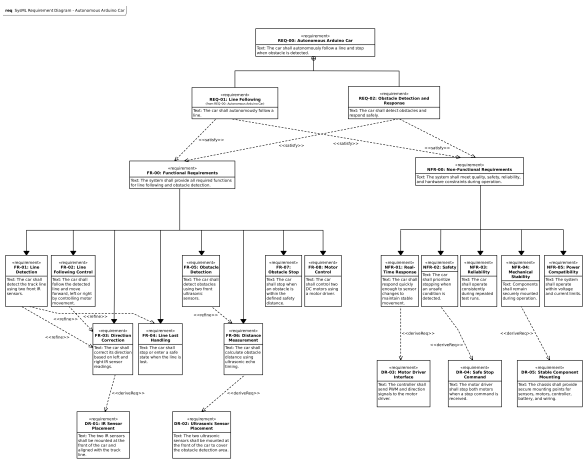


Figure 1. Requirement Diagram of the Autonomous Arduino Car

The requirement diagram in Figure 1 shows the main system requirements for the autonomous Arduino car. The top-level requirement is REQ-00: Autonomous Arduino Car, which states that the car shall autonomously follow a line and stop when an obstacle is detected. This is divided into two main requirements: Line Following and Obstacle Detection and Response. The functional requirements describe the required system behavior, such as line detection, direction correction, distance measurement, obstacle stopping, and motor control. The non-functional requirements describe quality and constraint aspects, such as real-time response, safety, reliability, mechanical stability, and power compatibility.

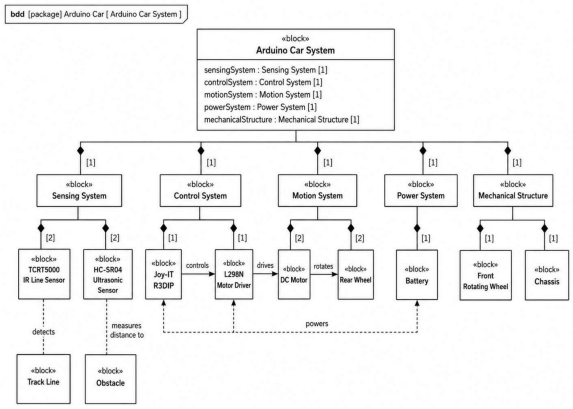


Figure 2. Block Definition Diagram of the Arduino Car System

The Block Definition Diagram in Figure 2 shows the main structure of the system. The complete Arduino car is divided into five subsystems: Sensing System, Control System, Motion System, Power System, and Mechanical Structure. The sensing system contains IR and ultrasonic sensors. The control system contains the controller board and motor driver. The motion system contains the DC motors and wheels. The power system contains the battery, and the mechanical structure contains the chassis and the front rotating wheel.

The engineering logic follows the flow from sensing to decision-making and movement. The sensors collect information from the environment. The controller processes the sensor values and decides whether the car should move forward, turn, stop, or react to an obstacle. The motor driver receives the control signals and drives the motors. The battery supplies the required power, while the chassis and holders keep the components securely mounted. The electronic design was planned to connect Arduino, motor driver, motors, battery, and switch safely and correctly.

2.1 Mechanical Prototyping Design Approach

2.1.1 System Overview and Objectives

To ensure smooth operation of the line-following robot, we designed 3D-printed mount for each major component: the battery, the ultrasonic sensor, the IR sensor, and the motor. Of these four, the IR sensor mount and the motor mount presented the greatest design challenges. The IR sensor lies at the heart of the vehicle's line-following capability — its readings drive every downstream decision in the control logic and every subsequent motion of the vehicle — so the rigidity and stability of its mount were critical to overall performance. The motor mount, in turn, had to keep the vehicle steady through both straight-line travel and

turns, preventing vibration or shifting from degrading driving performance. Across all four components, our design work centered on building a stable supporting structure while preserving room for future adjustment and refinement.

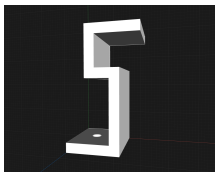


Figure 3. 3D-printed infrared sensor holder.

2.1.2 Battery Mount and Ultrasonic Sensor Mount

Compared with the IR sensor mount and the motor mount, the battery mount and the ultrasonic sensor mount were relatively straightforward and required only a handful of revisions before reaching their final form. For the battery mount, we placed one bracket on each side of the battery to hold it in place, with the goal of making the battery easy to slide out and replace. Since the breadboard is slightly wider than the battery itself, we cut a matching groove into the mount (Figure 4) so the breadboard could seat firmly into position. In testing, this design let us remove and reinsert the battery with ease while still holding both the battery and the breadboard securely in place. For the ultrasonic sensor, our goal was to mount it securely to the chassis without the mount itself interfering with signal transmission or reception. We designed an L-shaped bracket with two openings sized to match the transmitter and receiver modules of the sensor (Figure 5). This kept the sensor firmly fixed in place while leaving its signal path completely unobstructed.

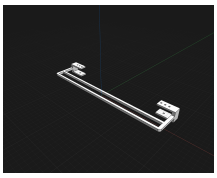


Figure 4. Battery mount with side brackets and breadboard groove.

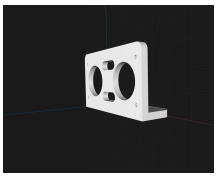


Figure 5. L-shaped ultrasonic sensor mount.

2.1.3 IR Sensor

The IR sensor mount was the project's most challenging design task. To combine rigidity with adjustable sensor spacing, we divided it into two parts: an IR sensor bracket (Figure 3) and a sliding base (Figure 6). A central channel allows the bracket to move laterally, enabling sensor spacing adjustments during testing without permanently fixing the mount to the chassis.

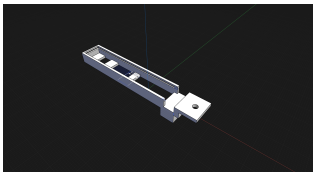


Figure 6. Sliding base for the IR sensor bracket.

The sensor bracket itself went through several rounds of iteration. The first version was an L-shaped bracket (Figure 7), but because the IR sensor's weight was unevenly distributed, the bracket tilted downward under the one-sided load, throwing off the detection angle. We redesigned it as a T-shape (Figure 8): one arm held the sensor while the other was fastened to the chassis to counterbalance the load.

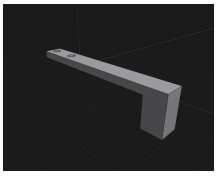


Figure 7. First iteration: L-shaped IR bracket.

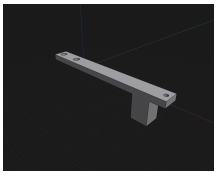


Figure 8. Second iteration: T-shaped IR bracket.

This introduced a second problem: the sensor's protruding pins prevented the bracket from sitting flush. We reshaped it into a small enclosure (Figure 9) with underside cutouts to clear each component. When assembled, we mounted it to the underside of the chassis (Figure 10), which kept the sensor level but placed it too close to the ground and degraded detection accuracy.

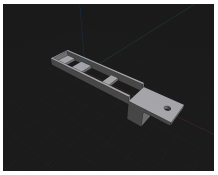


Figure 9. Third iteration: enclosure-style bracket.

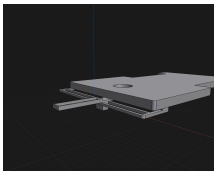


Figure 10. Enclosure mounted under the chassis.

In the final version, we relocated the sliding base to the center of the chassis plate (Figure 11) and redesigned the bracket to follow the chassis contour, fastening it to the top rather than the underside. This resolved the weight imbalance, pin interference, and excessive ride height.

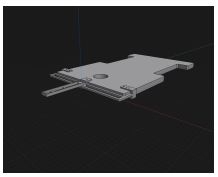


Figure 11. Final IR sensor mount on the chassis plate.

2.1.4 Motor Mount

The motor mount holds the motor firmly to the chassis, preventing vibration from compromising driving stability. The first version (Figure 12) was overly complex, and housing protrusions made assembly difficult. We simplified it to a three-sided wrap-around bracket with a tight tolerance fit. That version used only two fastening points and loosened under vibration, so we extended the base and switched to four-point fastening (Figure 13), keeping the motor stable in both forward driving and turning.

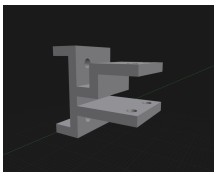


Figure 12. First iteration: complex motor mount.

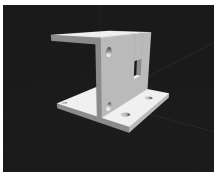


Figure 13. Final motor mount, four-point fastening.

3. Hardware and Software Implementation

3.1 System Overview

The autonomous vehicle combines an Arduino microcontroller, a motor driver, two DC motors, two infrared sensors, two ultrasonic sensors, and a battery-powered chassis.

Custom 3D-printed holders secure the main components and keep the sensors in stable positions, improving the consistency of the measurements.

The Arduino reads the sensor values and executes the corresponding control logic. It then sends speed and direction commands to the motor driver, which controls the two motors independently using pulse-width modulation.

This allows the vehicle to move forward, reverse, follow curves, and perform turning manoeuvres.

Sensors → Implemented code → Motor driver → DC motors

3.2 Line-Following Logic

Two infrared sensors are mounted at the front of the vehicle and positioned close to each other. This narrow spacing was selected because the black line has a thickness of approximately 2.5 cm.

The placement keeps the sensors close to the line and allows small deviations to be detected before the vehicle moves completely away from the track. The implemented code continuously reads both sensors and evaluates three main cases.

3.2.1 Line Lost: White–White

When both sensors detect the white surface, the black line is temporarily lost.

The implemented code stores the side on which the line was last detected. The vehicle then searches in the same direction by applying different speeds to the left and right motors. This memory-based movement prevents random turning and improves recovery during curves.

3.2.2 Line Edge: White–Black or Black–White

When one sensor detects the black line and the other detects the white surface, the vehicle is positioned near one edge of the line.

The sensor combination determines the required correction direction. One motor is driven faster while the other is slowed down, causing the vehicle to curve back toward the line.

A proportional-derivative correction is used to produce smoother movement. The proportional part responds to the current directional error, while the derivative part reduces sudden changes and jerking.

The detected side is also stored so that it can be used if the line is later lost.

3.2.3 Line Centred: Black–Black

When both sensors detect the black line, the vehicle is correctly positioned on the track. No major directional correction is required, so both motors operate at approximately the same speed and the vehicle continues forward.

As shown in Figure 14, the final implementation keeps the vehicle aligned with the black line during forward movement. This demonstrates that the close sensor placement, continuous sensor feedback, and balanced motor control provide stable line-following behaviour.

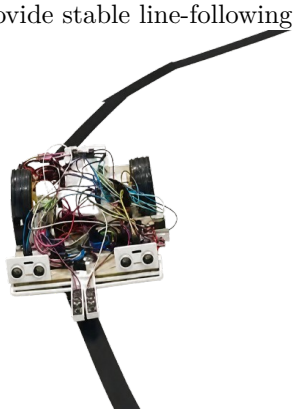


Figure 14. Vehicle following the black line.

3.3 Obstacle Detection

Two ultrasonic sensors are mounted at the front-left and front-right sides of the vehicle. Their wide placement provides broader obstacle coverage across the front of the vehicle.

Each sensor sends an ultrasonic pulse and measures the time required for the echo to return. The Arduino converts this time into a distance value.

The implemented code compares both readings and uses the smaller valid distance. Therefore, the vehicle reacts when either sensor detects a nearby obstacle.

When the distance falls below the defined threshold, the vehicle stops and takes additional measurements. The obstacle is accepted only when several readings confirm its presence. This reduces false reactions caused by temporary measurements or reflections.

3.4 First Obstacle Avoidance

The first confirmed obstacle activates the complete avoidance routine:

1. Stop and confirm the obstacle.
2. Reverse to create turning space.
3. Turn away from the obstacle.
4. Move forward beside the obstacle.
5. Turn back toward the track.
6. Search for the black line.
7. Resume normal line following.

The left and right ultrasonic readings are compared before the initial turn. This allows the implemented code to adjust the turning movement according to the position of the obstacle.

During recovery, the infrared sensors continuously search for the black line. When the line is detected,

the avoidance routine ends and normal line following resumes.

As shown in Figure 15, the vehicle successfully moves around the first obstacle and rejoins the black line. Obstacle detection is then briefly ignored to prevent the same obstacle from being detected again immediately.

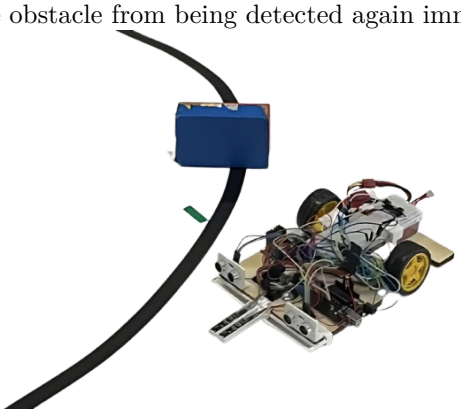


Figure 15. First obstacle avoidance and line rejoining.

3.5 Reverse Turn Logic

The implemented code records when the first obstacle has been avoided. This memory-based condition causes the second and every later obstacle to activate the reverse routine instead of repeating the complete avoidance sequence.

The vehicle stops, confirms the obstacle, reverses to create turning space, and performs a slow 180-degree turn. At the beginning of the turn, the infrared sensor readings are briefly ignored to prevent the original line from being detected too early.

After a minimum turning time, the sensors begin searching for the black line. The line must be detected in several consecutive readings before the turn is completed.

As shown in Figure 16, the vehicle performs a controlled 180-degree turn after detecting the second obstacle. Once the black line is confirmed, the vehicle stabilizes briefly and resumes normal line following in the opposite direction.

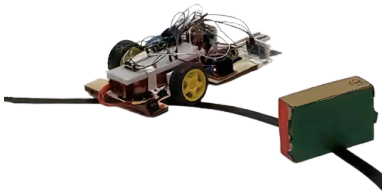


Figure 16. Controlled 180-degree turn and line rejoining after detection of the second obstacle.

4. Simulation and Testing Results

4.1 Tinkercad Simulation

Tinkercad was used to simulate the electronic system before the final assembly of the physical prototype. It helped the team understand the components and their connections, plan the Arduino pin assignments, reserve sufficient PWM pins for motor control, and create an initial wiring reference for the physical vehicle.

The virtual circuit also supported the early arrangement of the Arduino, breadboard, motor driver, battery, and sensors on the chassis. This made it easier to estimate the required space, connection directions, and cable routing before assembly.

Because the Tinkercad library did not contain all the required components, equivalent elements were used. Switches represented the digital black–white states of the infrared sensors, while an available integrated circuit represented the motor-control behaviour. A separate motor-driver circuit was also prepared to study the power connections between the battery, Arduino, motor driver, and DC motors.

4.2 Hardware Testing

4.2.1 Infrared Sensor Calibration

The infrared sensors were calibrated at their actual mounting height using their onboard potentiometers. Each sensor was tested over both the black line and the white surface until the two states were detected reliably. This adjustment minimized unstable readings and improved the consistency of the line-following behaviour.

4.2.2 Ultrasonic Sensor Testing

The ultrasonic sensors were tested using a simple distance-measurement program. Obstacles were positioned at different distances and angles, and the values were observed through the serial monitor. The tests helped determine the useful detection range and a suitable mounting position that provided front coverage without frequently detecting nearby walls.

4.2.3 Component Verification During Development

Component testing remained important after the complete vehicle had been assembled. During one test, the line-following behaviour became unstable and the motors appeared to respond incorrectly. The motors were tested in the forward and reverse directions, followed by separate tests of the ultrasonic and infrared sensors.

The motor system operated correctly, while the problem was traced to an incorrect infrared-sensor reading. This procedure demonstrated the importance of testing the components individually before making unnecessary changes to the control software.

5. Challenges Encountered and Solutions

5.1 Mechanical Design Challenges

Challenge: The main mechanical challenge was to design component holders that were stable and functional while requiring minimal printing time and material. The most demanding parts were the motor holder and the adjustable infrared-sensor holder.

Solution: Simple supports were used for components such as the battery and breadboard instead of large printed enclosures. The motor holder was designed to prevent movement during driving, line corrections, reversing, and turning. Because the available ground clearance was limited, the motors were mounted on the

upper side of the chassis while maintaining stable wheel alignment and contact with the ground.

The infrared-sensor holder was designed to allow horizontal adjustment according to the line width and the selected line-following logic. At the same time, it held the sensors firmly to prevent vibration or position changes that could cause incorrect readings.

The cables were also routed away from the front of the vehicle so that they would not enter the ultrasonic sensors' field of view and produce false measurements.

5.2 Software Challenges

5.2.1 Line-Following Stability

Challenge: The initial line-following program detected the track correctly, but the vehicle repeatedly oscillated between the left and right sides. Strong corrections caused abrupt movement, while weak corrections reacted too slowly on curves.

Solution: A proportional–derivative controller was used to produce smoother corrections. The proportional term reacted to the current directional error, while the derivative term reduced sudden changes and jerking.

The controller gains were tuned together with the main motor speed and the maximum correction value. A small difference between the left and right motor commands was also introduced to compensate for unequal motor responses. These adjustments improved stability and reduced repeated left–right oscillation.

5.2.2 Obstacle-Avoidance Reliability

Challenge: The main challenge was to develop an obstacle-avoidance logic that could bypass an obstacle correctly at different track locations and reliably recover the black line.

Solution: The avoidance routine was changed from a completely predefined sequence to a combination of adaptive, timed, and sensor-controlled movements. The left and right ultrasonic readings were compared before the initial turn, allowing the turning movement to be adjusted according to the relative obstacle position.

The initial movements remained time-based to reverse, create space, and pass around the obstacle. However, the final recovery movements continuously checked the infrared sensors instead of relying only on fixed delays.

A forced-recovery curve was added when the normal search did not locate the line. The `millis()` function allowed the program to continue reading the infrared sensors during the manoeuvre. As soon as the black line was detected, the avoidance routine stopped immediately and normal line following resumed. This prevented the vehicle from passing over the line while the recovery movement was still active.

6. Conclusion and Future Improvements

The autonomous Arduino car successfully demonstrated the main objectives of the project: line following, obstacle detection and avoidance, and autonomous turning behaviour. The integration of infrared sensors, ultrasonic sensors, DC motors, a motor driver, and the Arduino controller created a functional prototype capable of responding to different environmental conditions. The project also showed the importance of sensor positioning, mechanical stability, and careful adjustment of the control parameters for achieving reliable vehicle movement.

Future improvements could focus on increasing the accuracy and robustness of the sensing system. Adding a third infrared sensor would provide more detailed information about the position of the line and could improve performance on curves, intersections, and situations where the line is temporarily lost. A colour sensor could also allow the vehicle to recognise coloured markers and react to predefined commands, such as stopping, changing speed, or selecting a route. Further development could include PID-based steering control, additional ultrasonic sensors, wheel encoders, and improved obstacle-avoidance logic to achieve smoother and more precise autonomous driving.